International Journal on Digital Libraries (2026) 27:11
https://doi.org/10.1007/s00799-026-00447-x

A fragment-based model and annotation environment for discontinuous and overlapping terms

Giorgio Maria Di Nunzio[1] · Federica Vezzani[2]

Received: 28 April 2026 / Revised: 9 July 2026 / Accepted: 13 July 2026

Abstract

Terminology and entity annotation tasks in digital-library and corpus-oriented workflows often involve expressions that are discontinuous, overlapping, or partially shared. While several existing annotation environments support span-based and relation-based annotation, discontinuous structures are frequently represented indirectly through workaround mechanisms, such as linking independent spans via auxiliary relations. This may increase annotation complexity and reduce the transparency of the resulting annotation objects. This paper introduces Annotaterm, a lightweight web-based annotation environment designed to support the direct annotation of contiguous, discontinuous, and overlapping terms through a fragment-based interaction model. This model supports simple, composite, and overlapping annotations in a uniform and inspectable way. A preliminary user evaluation with 14 participants suggests that the tool is perceived as suitable for terminology-oriented annotation tasks, particularly for handling overlapping and discontinuous expressions, while also identifying areas for future refinement. The proposed approach contributes both a conceptual model and a practical annotation environment for digital-library and terminology-oriented workflows requiring transparent handling of complex textual structures.

Keywords Term extraction · Discontinuous annotation · Digital libraries

1 Introduction

Text annotation is a foundational activity in the creation of high-quality datasets for natural language processing (NLP), including named entity recognition, term extraction, and relation identification. Annotated corpora support the training and evaluation of supervised models, but they also play a broader role in the organization, interpretation, and reuse of knowledge within digital libraries and domain-specific repositories. In this sense, annotation is not only a preparatory step for downstream NLP, but also a form of scholarly data curation that contributes to semantic indexing, metadata enrichment, and interoperability across research infrastructures.

This broader perspective is especially relevant in digital-library and digital-humanities settings, where annotations function as structured interpretative layers linking textual evidence to concepts, terminology resources, and external knowledge resources [1]. As discussed by [2], annotation practices support not only retrieval and analysis but also long-term preservation and reuse. Consequently, annotation environments should be regarded as part of the scholarly infrastructure through which linguistic and conceptual knowledge is created, maintained, and made reusable. Their expressive power therefore matters: limitations in annotation models and interfaces directly affect the quality and granularity of the resources that digital libraries are able to curate.

One persistent limitation concerns the representation of discontinuous and overlapping multi-word expressions. While contiguous entities and terms such as "entity recognition" are straightforward to annotate, many domain-relevant expressions are distributed across non-adjacent textual fragments or share lexical material with other candidate units. For example, in the phrase "entity recognition and linking," the terms "entity recognition" and "entity linking" overlap on a shared component and cannot be adequately represented as independent contiguous spans. Such phenomena

✉ Giorgio Maria Di Nunzio
giorgiomaria.dinunzio@unipd.it

Federica Vezzani
federica.vezzani@unipd.it

[1] Department of Information Engineering, University of Padua, Via Gradenigo 6a, 35131 Padova, Italy

[2] Department of Linguistic and Literary Studies, University of Padua, Via Vendramini 13, 35137 Padova, Italy

Published online: 01 August 2026

are common in domains such as biomedicine, terminology, lexicography, corpus annotation, and the digital humanities, yet they remain difficult to encode in many current annotation environments.

Recent reviews show that most annotation tools either do not support discontinuous spans or rely on indirect workarounds, such as linking separate spans through relations, rather than treating fragmented expressions as first-class annotation objects [3–5]. Although some systems, such as TextAE [6], offer explicit support for non-contiguous annotation, the available landscape still provides limited support for workflows that combine discontinuous named entities, multi-word terms, and lightweight exportable representations within a single environment. This issue is particularly relevant for terminology-oriented annotation, which has received comparatively less attention than NER in the design of annotation platforms [7]. As a result, scholars, curators, and dataset builders often face substantial practical barriers when constructing terminology resources and databases of named entities in a consistent and reusable way.

This article addresses these limitations by proposing a fragment-based approach to text annotation in which each component of a complex expression is represented as an explicit annotation unit linked to a shared identifier. Building on this idea, we present Annotaterm, a lightweight web-based system developed in R Shiny[1] for the annotation of discontinuous named entities and multi-word terms in digital-library contexts. This work extends a preliminary system paper presented at IRCDL 2026 [8], which introduced the initial motivation and prototype of Annotaterm. The present article substantially expands that contribution in four directions: it refines the conceptual framing of discontinuous and composite annotation, formalizes the fragment-based data model underlying the system, situates Annotaterm more explicitly with respect to existing annotation tools, and adds a preliminary user evaluation of the approach. The overall aim is not only to introduce a tool, but to contribute a reusable annotation model and workflow for the curation of structured linguistic and terminology resources that can support semantic enrichment, interoperability, and research data reuse.

The remainder of the paper is organized as follows. Section 2 reviews existing annotation environments with particular attention to their support for discontinuous and overlapping annotations, highlighting the motivations for the proposed approach. Section 3 derives the functional requirements that guided the design of Annotaterm. Section 4 introduces the fragment-based annotation model and discusses its representation principles. Section 5 presents the architecture, data model, annotation workflow, and implementation of Annotaterm. Section 6 reports the preliminary user evaluation and discusses both quantitative and qualitative findings. Finally, Sect. 7 summarizes the main contributions and outlines directions for future work.

[1] https://shiny.posit.co/

2 Background and related work

Text annotation is a foundational process in the creation of high-quality datasets for named-entity recognition (NER), term extraction, and related information-extraction tasks. Recent advances in automatic NER have shown that discontinuous multi-span entities can be detected effectively through neural approaches, confirming that such mentions are not marginal phenomena but viable targets for computational modeling [9]. However, the availability of expressive automatic models does not automatically translate into equally expressive manual annotation environments. Creating ground-truth datasets for discontinuous and overlapping expressions remains difficult in practice, because annotators often lack interfaces that allow them to select, visualize, and manage multiple non-adjacent spans as parts of a single annotation object.

This challenge arises in several domains, including biomedicine, legal and administrative text, terminology work, and scholarly corpora. It is particularly evident when candidate expressions are not realized as simple contiguous spans, but as combinations of fragments that are separated in the text or shared across multiple units. While contiguous entities and terms such as “metadata creation” are straightforward to annotate, many domain-relevant expressions are distributed across non-adjacent textual fragments or share lexical material with other candidate units. For example, in the phrase “metadata creation and enrichment,” the terms “metadata creation” and “metadata enrichment” overlap on a shared component and cannot be adequately represented as independent contiguous spans. Such cases illustrate a broader tension between the complexity of the linguistic phenomena to be annotated and the limitations of many current annotation interfaces.

Recent reviews have clarified different aspects of this problem. The systematic review by [3] offers a comprehensive account of discontinuous named entities in biomedical and clinical corpora, focusing primarily on corpus properties and computational approaches. However, it does not examine in detail how annotation interfaces support the manual creation and correction of such structures. Conversely, the comparative review by [4] surveys a broad range of semantic and non-semantic annotation tools, with particular attention to usability and interoperability, but without a specific focus on discontinuous or overlapping span annotation. Taken together, these studies show that both the phenomenon and the annotation-tool ecosystem are well recognized, yet the specific question of how discontinuous and composite expressions can be supported in practical annotation workflows remains insufficiently addressed.

Within the current landscape, most annotation environments still assume that entities correspond to contiguous spans. Tools such as BRAT [10] and INCEpTION [11] support rich annotation workflows, but discontinuous mentions are typically handled through indirect workarounds, for example by linking separate spans through relations. While such solutions may be sufficient for some corpus-engineering scenarios, they can be cumbersome when annotators need to identify and manipulate fragmented expressions directly. In these cases, representational capacity alone is not enough: the usability of the annotation workflow becomes equally important.

Other systems provide more explicit support for multi-span structures. MedTator [12], for example, relies on the BioC format [13], which allows several offset pairs to be associated with a single annotation. This improves representational expressiveness, but does not by itself guarantee an efficient interaction model for selecting, editing, and validating fragmented spans. Similarly, Metatron [14] supports span-, relation-, and document-level annotation in biomedical settings, yet currently treats non-contiguous mentions indirectly rather than through native multi-span interaction.

TextAE [6] is one of the few tools to introduce an explicit non-contiguous annotation mode. Its extension of the BRAT visualization paradigm allows users to associate multiple spans with one entity and to export them in BioC format. This represents an important step forward and shows that discontinuous annotation can be supported natively within a web-based environment. At the same time, TextAE remains primarily entity-centric and offset-based, and offers limited support for workflows oriented toward terminology curation, fragment aggregation, and lightweight tabular representations suitable for iterative corpus-building and resource maintenance.

This last point is especially relevant for term extraction and terminology-oriented annotation. Compared with NER, terminology work has received less attention in the design of annotation platforms [7]. Yet the identification of domain-specific lexical units often requires attention to recurrent multi-word patterns, formal variants, and partial lexical overlap across documents. For digital libraries and terminology resources, the issue is therefore not only whether discontinuous spans can be encoded, but also whether they can be represented in a way that supports reuse, export, and integration with downstream curation workflows.

The gap identified in this article lies at the intersection of these requirements. Existing tools either privilege contiguous span annotation, rely on workaround-based representations for discontinuous cases, or provide only limited support for terminology-oriented workflows. In response, Annotaterm adopts a fragment-based representation in which each segment is treated as a first-class annotation unit linked to a shared identifier. This design aims to support both named-entity and term annotation while enabling simpler exports and closer integration with terminology extraction and digital-library curation pipelines.

3 Functional requirements and design challenges

As the previous section has shown, the annotation of discontinuous and overlapping expressions raises difficulties that are not limited to one technical layer of the annotation process. The challenge is simultaneously interactive, representational, and organizational. At the interaction level, annotators need mechanisms for selecting, linking, visualizing, modifying, and deleting multiple fragments that belong to one single concept. At the representational level, the underlying schema must preserve these structures without flattening them into contiguous spans or dispersing them across loosely connected objects. At the workflow level, the resulting annotations should remain interoperable, reusable, and suitable for terminology-oriented curation tasks in digital-library settings.

Existing annotation environments only partially address these requirements. General-purpose platforms such as BRAT and INCEpTION provide workaround-based solutions
through relation linking, but these often require annotators to create several independent objects and then connect them explicitly. This increases interaction complexity and makes the resulting representation harder to inspect and maintain. By contrast, a fragment-oriented workflow should allow users to construct complex annotations incrementally through direct selection of textual fragments, while preserving a coherent representation of the resulting object.

For this reason, the design of an annotation system for discontinuous entities and terms must be guided by a clear set of functional requirements derived from both interface limitations and data-model constraints.

3.1 Interaction challenges

Most annotation tools assume that each entity corresponds to one contiguous span of text. While this assumption simplifies the interface, it fails to capture linguistic patterns such as coordination, insertion, and partial lexical overlap. For example, in the phrase “entity recognition and linking,” the expressions “entity recognition” and “entity linking” share one lexical component and cannot be represented adequately as two independent contiguous spans without losing structural information about their overlap.

A second challenge concerns the visual and operational management of fragmented annotations. When an annotation consists of multiple segments, the interface should make

their relationship immediately understandable to the annotator. This includes not only consistent visual highlighting, but also basic editing operations such as extending, deleting, or correcting fragments without forcing the user to reconstruct the annotation from scratch. In this respect, the key issue is not only whether a system can encode discontinuity, but whether it can support it through a usable workflow.

3.2 Representation and schema challenges

The internal representation of annotations imposes further constraints. Token-labeling schemes such as BIO or IOB2 cannot directly encode multiple disjoint spans belonging to the same entity [15]. Similarly, many standoff models assume one offset pair for each annotation object, making discontinuous structures difficult to represent without auxiliary relations or custom extensions.

Formats such as BioC [16] address this limitation by allowing multiple <*location*> elements within a single annotation. This provides an interoperable way to encode discontinuous mentions, but does not by itself solve the interaction problem: an expressive schema is only useful if the annotation interface allows users to create and inspect such structures efficiently.

A related issue concerns annotation integrity. A system supporting fragment-based annotation should prevent inconsistent states such as redundant duplicates, accidental one-fragment composites when these are not conceptually distinct, or composite structures that cannot be edited transparently. These constraints are part of the data model as much as of the user interface.

3.3 Workflow challenges for terminology annotation

The problem becomes more complex when terminology-oriented annotation is considered alongside NER. Unlike named entities, terms are not limited to proper names and often appear as multi-word expressions, morpho-syntactic patterns, abbreviations, or lexical variants. Annotating terms may therefore require more flexible category structures, support for recurring variants, and representations that can be exported into tabular or structured formats suitable for terminology management and digital-library curation.

For this reason, a system intended for both named entities and terms should not merely support discontinuous spans. It should also provide a representation that remains lightweight, inspectable, and compatible with downstream workflows such as terminology extraction, corpus maintenance, and semantic enrichment.

3.4 Functional requirements

Based on these observations, the following requirements guide the design of the proposed system.

1. Fragment-based selection. Annotators must be able to select multiple non-contiguous spans and associate them with one annotation object through direct interaction.
2. Coherent visualization. Fragments belonging to the same annotation should be rendered consistently, so that their unity is immediately visible and cognitively manageable.
3. Editable composite structures. Annotators should be able to inspect, correct, delete, and revise fragmented annotations without reconstructing them manually.
4. Flexible internal representation. The data model must preserve multiple offsets or token indices for one annotation while remaining simple enough for storage, inspection, and reuse.
5. Integrity constraints. The system should prevent redundant or inconsistent annotation states, including uncontrolled duplication and ambiguous handling of simple versus composite cases.
6. Terminology-oriented support. The annotation model should accommodate terms as well as named entities, including multi-word terms, label hierarchies, and variant-oriented workflows.
7. Interoperability. Import and export mechanisms should preserve discontinuous structures across reusable formats such as BioC and tabular representations.
8. Lightweight deployment. The system should be web-accessible, easy to deploy, and suitable for small- to medium-scale digital-library annotation projects.

Not all requirements have the same scope. The first five are foundational, because they define the minimum conditions under which discontinuous and overlapping annotation can be performed coherently. The remaining requirements concern extension toward broader terminology and digital-library workflows.

These requirements lead to two design questions that structure the remainder of the article:

- How can a web-based interface support fragment-based annotation of discontinuous and overlapping expressions without imposing excessive cognitive or operational burden on annotators?
- How can a fragment-based representation balance expressiveness, interoperability, and reuse across NLP and digital-library curation workflows?

The next section presents Annotaterm, a lightweight prototype developed in the R Shiny framework, and shows how its design operationalizes these requirements through a fragment-based annotation model.

4 A fragment-based model for discontinuous and overlapping annotation

The design of an annotation system with the functional requirements expressed in the previous section should be grounded in a fragment-based representation of textual annotation. The purpose of this model is to support expressions that cannot be captured adequately as single contiguous spans, including discontinuous, overlapping, and partially shared multi-word units. Rather than treating these cases as exceptions to a contiguous-span norm, the model adopts fragments as the basic units from which more complex annotations are constructed.

4.1 Annotation objects and fragments

We define an *annotation object* as a labeled textual unit associated with one or more fragments in a document. A *fragment* is a contiguous segment of text identified by its surface form and positional information, such as character offsets or token indices. An annotation object may therefore correspond either to a single fragment or to a set of fragments linked under a shared identifier.

This distinction allows the model to separate the conceptual identity of an annotation from its textual realization. For example, a contiguous term such as "entity recognition" is represented as one annotation object with one fragment. By contrast, in the phrase "entity recognition and linking," the annotation object corresponding to "entity linking" is represented through two fragments, namely "entity" and "linking," linked under the same identifier. In this way, contiguous and non-contiguous annotations can be treated uniformly within the same framework.

4.2 Simple, composite, and overlapping cases

The model distinguishes between *simple* annotations, which consist of a single fragment, and *composite* annotations, which consist of two or more fragments. Composite annotations are especially useful when an expression is distributed across separate portions of the text or when multiple candidate expressions share lexical material.

Consider the phrase "entity recognition and linking." Within a traditional span-based model, the expressions "entity recognition" and "entity linking" are difficult to encode without either duplicating material or introducing indirect relational workarounds. In the fragment-based model, each annotation object can be associated with the relevant fragments while preserving the fact that some textual material may participate in more than one annotated unit. This makes the representation suitable not only for discontinuity in the strict sense, but also for overlap and partial lexical sharing.

4.3 Representation principles

The proposed model follows four principles.

1. *Fragments are first-class units.* Each selected segment is represented explicitly rather than being hidden inside a nested or purely relational structure.
2. *Annotation identity is independent of fragment count.* A single annotation object may contain one fragment or several fragments without changing its conceptual status.
3. *Complexity is handled through aggregation rather than indirection.* Multi-part expressions are constructed by aggregating fragments under a shared identifier, rather than by creating multiple independent entities linked through auxiliary relations.
4. *The representation should remain inspectable and exportable.* The internal structure of annotations should support transparent tabular views and straightforward transformation into interoperable formats.

These principles are particularly important in terminology-oriented workflows, where annotators often need to inspect recurring multi-word patterns, compare formal variants, and maintain resources that remain understandable outside the annotation environment itself.

4.4 Advantages over workaround-based representations

Many existing annotation environments represent discontinuous structures only indirectly. A common strategy is to annotate each fragment separately and then connect the resulting spans through a relation. Although this makes the phenomenon encodable, it also shifts complexity onto the annotator and weakens the unity of the resulting annotation object.

The fragment-based model proposed here adopts a different perspective. Instead of treating fragmentation as a relation between independent entities, it treats fragmentation as an internal property of one annotation object. This has at least three advantages. First, it reduces conceptual distance between what the annotator perceives and what the system stores. Second, it facilitates inspection and editing, because all fragments belonging to one annotation remain grouped. Third, it supports simpler export strategies for downstream workflows in terminology management, corpus curation, and digital-library enrichment.

5 Annotaterm: design and model

To operationalize the requirements discussed in Sect. 3, we developed Annotaterm, a lightweight web-based annotation

environment implemented in R Shiny. The system is designed to support the annotation of both contiguous and discontinuous expressions through a fragment-based interaction model, while remaining easy to deploy and inspect in small- to medium-scale annotation settings. The source code is openly available on GitHub.[2] A screenshot of the main interaction panel is shown in Fig. 1.

Rather than treating discontinuous mentions as exceptional cases represented through auxiliary relations, Annotaterm models them through the direct aggregation of textual fragments under a shared annotation identifier. This design makes fragmented expressions visible and editable as unified objects, while preserving a tabular internal structure suitable for export, inspection, and downstream reuse.

5.1 System overview

The system follows a client–server architecture in which all annotation actions are performed through the web browser and managed reactively by the Shiny server. The interface includes a central text area for document display and text selection, together with side-panel controls for annotation management and summary tables for inspection of the stored annotations. User selections are captured through a lightweight JavaScript layer and transmitted to the server through Shiny's reactive input mechanism, where they are transformed into annotation records and rendered back into the interface.

This architecture supports immediate feedback during annotation: each change in the stored data triggers a recomputation of the highlighted fragments and an update of the annotation tables. The result is a workflow in which annotation, inspection, and correction remain closely integrated.

5.2 Fragment-based data model

The central design choice of Annotaterm is the adoption of a fragment-based representation. Instead of storing a discontinuous annotation as a single object containing a nested list of offsets, the system records each fragment as an explicit annotation unit linked to a shared identifier. This representation treats fragments as first-class components of a larger annotation object and makes it possible to distinguish clearly between the annotation as a conceptual unit and its textual realization as one or more segments.

Each annotation includes a label, a type, and a set of associated fragments. For each fragment, the system stores the selected text and its character offsets within the document. Contiguous annotations are represented as single-fragment cases, whereas discontinuous annotations are represented as multi-fragment cases associated with the same identifier. This model preserves the internal structure of complex expressions while remaining compatible with tabular data processing and relational storage.

A further advantage of the fragment-based approach is that it supports explicit integrity constraints. Because fragments are stored individually but linked systematically, the system can detect redundant duplicates, prevent degenerate composite structures, and normalize simple and multi-fragment annotations consistently. These constraints are important not only for correctness, but also for transparent editing and export.

5.3 Annotation workflow

Annotaterm is designed around a direct fragment-selection workflow. Users select text within the document display and can store it either as a simple annotation or as part of a composite annotation. In the composite workflow, successive fragment selections are aggregated before being saved under a shared label. The system then updates both the visual highlights in the text and the tabular summary of the stored annotations.

Fragments belonging to the same annotation are rendered consistently through shared visual cues, allowing annotators to recognize them as parts of one object. The workflow also includes annotation-management operations for inspecting stored annotations and revising them when needed. This aspect is especially important for discontinuous and overlapping cases, in which the ability to correct, remove, or normalize fragment structures is essential for practical use.

The resulting interaction model differs from workaround-based approaches in which discontinuous mentions are reconstructed indirectly through separate entities and explicit relation links. In Annotaterm, complexity is handled at the level of fragment aggregation rather than by requiring annotators to build and maintain auxiliary relational structures.

5.4 Implementation and deployment

The current implementation remains deliberately lightweight. Annotaterm has been developed in fewer than 300 lines of code and relies on standard R Shiny components, minimal JavaScript for text selection, and reactive data structures for annotation storage and rendering. This choice facilitates rapid deployment, local adaptation, and integration with existing R-based data-processing workflows.

The use of a tabular internal representation also supports transparent export and inspection. Because each fragment corresponds to one record linked to an annotation identifier, the resulting data can be exported in simple structured formats such as CSV or JSON and adapted to more specialized representations as needed. This makes the system particularly

[2] https://github.com/gmdn/IJDL2026/tree/main/Annotaterm

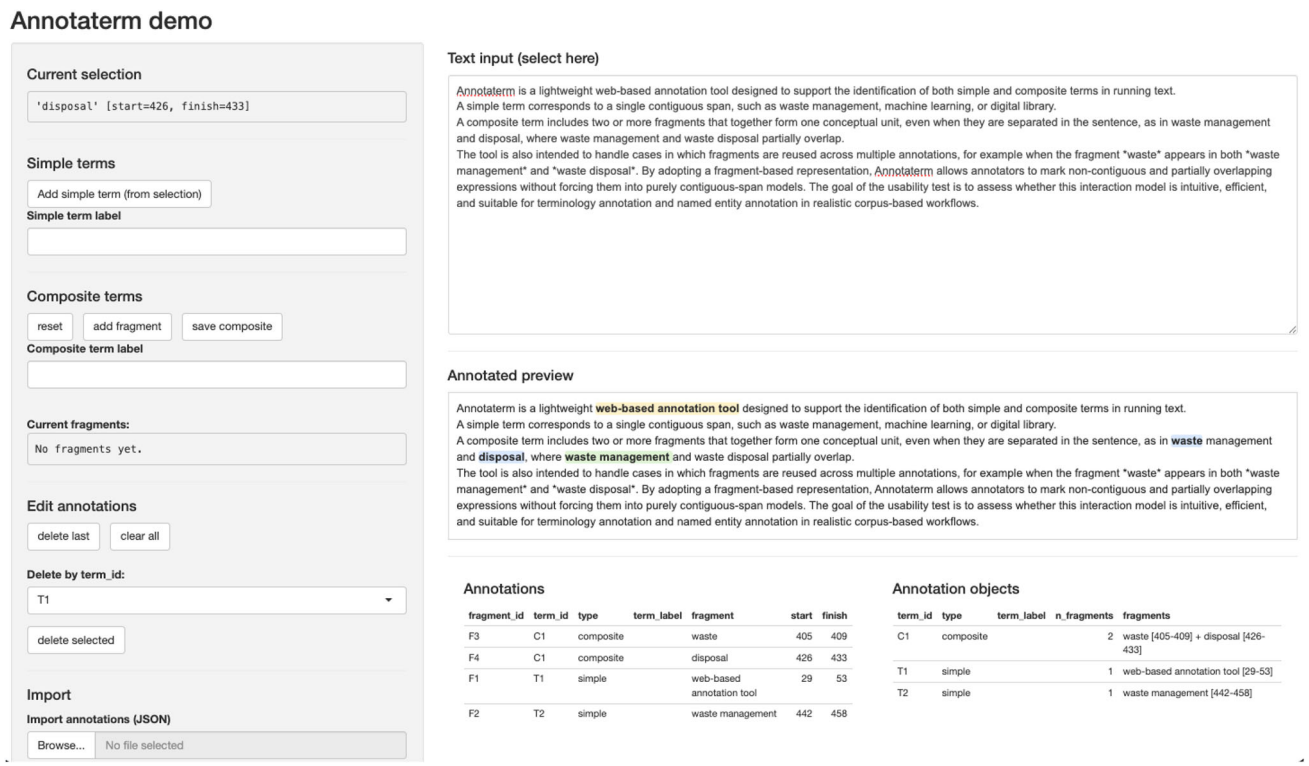

Fig. 1 A screenshot of the annotaterm tool

suitable for experimental annotation projects, terminology-oriented corpus building, and digital-library workflows in which portability and inspectability are important.

In the current implementation, the input text is intended to remain stable during an annotation session. If the text is modified after annotations have been created, users are warned that existing character offsets may no longer correspond to the updated text; future versions will support locking the input text once annotation begins.

5.5 Comparison with existing annotation environments

Existing annotation environments differ not only in their feature sets, but also in the way they conceptualize discontinuous annotation. General-purpose tools such as BRAT [10] and INCEpTION [11] provide rich support for span and relation annotation, but they are primarily designed around annotation objects that are created as spans and then connected through additional structures. This makes them highly flexible, yet less direct for workflows in which annotators need to construct and inspect discontinuous expressions as unified objects from the outset.

TextAE [6] is the most relevant point of comparison, since it explicitly supports non-contiguous entities by allowing users to add multiple subspans to the same annotation. This is an important advance over workaround-based approaches and demonstrates that non-contiguous annotation can be implemented effectively in a web environment. However, TextAE remains largely entity-centric and offset-oriented, with its design tightly connected to the PubAnnotation ecosystem. By contrast, the model proposed in this article treats fragments as first-class annotation units and emphasizes lightweight, inspectable representations suitable not only for entity annotation, but also for terminology-oriented curation and tabular export workflows.

MedTator [12] and MetaTron [14] illustrate two further directions in the design space. MedTator emphasizes lightweight corpus-development workflows, annotation export, and adjudication, while MetaTron integrates collaborative annotation and automatic prediction support in the biomedical domain. These systems show that usability, collaboration, and workflow support are central concerns in annotation-tool design. At the same time, they do not place fragment aggregation at the center of the annotation model in the way proposed here.

The comparison therefore suggests that the main contribution of Annotaterm does not lie simply in supporting discontinuous spans. Rather, it lies in adopting a fragment-based representation in which discontinuity and overlap are modeled internally as properties of one annotation object, and in aligning that representation with terminology-oriented and digital-library annotation workflows (Table 1).

Table 1 Comparison of annotation environments with respect to discontinuous annotation and terminology-oriented workflows

Tool	Primary orientation	Support for discontinuity	Main interaction model	Main limitation for our use case
BRAT	General span/relation annotation	Indirect/workaround-based	Spans plus relations	Discontinuous structures are not treated as unified objects
INCEpTION	Semantic annotation platform	Indirect/workaround-based	Spans, layers, relations, assisted annotation	Powerful but heavier and less direct for fragment-oriented workflows
MedTator	Lightweight biomedical corpus development	Partial/format-supported	Span annotation with export/adjudication focus	Less focused on direct fragment aggregation for terminology workflows
MetaTron	Biomedical collaborative annotation	Limited support	Mention-, relation-, and document-level annotation	Strong collaboration features, but not centered on fragment-based discontinuous annotation
TextAE	Biomedical entity annotation	Explicit native support	Multi-subspan entity editing	Strongest comparator, but still mainly entity-centric and offset-oriented
Annotaterm	Entity and terminology annotation for digital-library workflows	Explicit fragment-based support	Direct fragment aggregation under shared identifier	Current scope lighter than full collaborative platforms

6 Evaluation

The evaluation was designed to assess Annotaterm as a lightweight annotation environment for discontinuous and overlapping expressions, with particular attention to the usability of the fragment-based workflow and to its adequacy for terminology-oriented and digital-library annotation tasks. Rather than benchmarking annotation speed or output quality against gold-standard corpora, the present study focuses on how users interact with the system, how clearly they understand the fragment-based model, and whether they perceive the interface as suitable for practical annotation scenarios.

6.1 Evaluation goals

The study addressed two main questions. First, can users understand and apply the fragment-based annotation workflow when working with both contiguous and discontinuous expressions? Second, does the interface provide a sufficiently clear and usable environment for annotation tasks involving multi-fragment and overlapping units?

The aim of the evaluation was therefore not to establish task performance in the narrow sense, but to examine whether the proposed design is understandable, usable, and perceived as appropriate for the intended annotation context.

6.2 Study design

We conducted a small-scale user study in which participants performed controlled annotation tasks on short texts containing contiguous, discontinuous, and overlapping expressions. The study was designed to observe interaction with the system under realistic annotation conditions while keeping the task manageable for participants with different levels of prior experience.

Participants were asked to identify and annotate relevant terms according to a short set of instructions. The selected material included examples of simple contiguous spans, discontinuous expressions, and partially overlapping multi-word units, exposing participants to the main phenomena that motivated the design of Annotaterm.

6.3 Participants

The study involved 14 participants enrolled in the Master's Degree programme in Modern Languages for Commu-

nication and International Cooperation. Participants were first- and second-year students attending a Terminography course and therefore had domain-specific familiarity with terminology-oriented annotation tasks.

All participants had at least some familiarity with annotation tools: 9 reported a *moderate* level of prior experience and 5 reported *limited* experience. No participant reported either *expert* or *no* prior experience. This variability reflects differences in prior practical exposure. Some participants had previously used the *MetaTron* annotation environment in the context of the GutBrainIE challenge[3] and the ATE-IT challenge,[4] while others had no direct experience with that platform or with similar collaborative annotation tasks. As a result, the participant group represented a relatively homogeneous population in terms of domain knowledge, but a more varied population with respect to practical familiarity with annotation environments.

Regarding task completion, 6 participants completed the annotation task fully, while 8 reported partial completion. No participant reported being unable to perform the task. Several partial completions were associated with device-specific issues, particularly when using tablets rather than desktop environments.

6.4 Materials and procedure

The evaluation used a small set of short texts selected to reflect the types of expressions that Annotaterm is intended to support. The material included examples of:

- Contiguous terms;
- Discontinuous expressions requiring multi-fragment annotation; and
- Overlapping or partially shared multi-word units.

Each participant completed the study individually. The procedure consisted of three phases.

First, participants received a short demonstration of the system, including the creation of both simple and composite annotations. Second, they performed the annotation task using Annotaterm. Third, after completing the task, they filled in a brief post-task questionnaire designed to collect feedback on usability, clarity of visualization, ease of interaction, and perceived suitability of the tool.

Participants were also invited to provide open comments on aspects of the interface they found particularly helpful or in need of improvement.

[3] https://hereditary.dei.unipd.it/challenges/gutbrainie/2025/

[4] https://nicolacirillo.github.io/ate-it/

6.5 Measures

The evaluation combined structured usability feedback with qualitative observations. Participants completed a short post-task questionnaire including Likert-scale items (1–5) on:

- Ease of understanding the interface;
- Ease of selecting fragments and creating annotations;
- Clarity of the distinction between simple and composite annotations;
- Ease of annotating overlapping or nested terms;
- Usefulness of the annotation tables and highlighted preview; and
- Overall suitability of the tool.

Open-ended responses were collected to identify recurring impressions, perceived strengths of the fragment-based model, and aspects of the interface requiring refinement.

The anonymized questionnaire responses and aggregated evaluation data are made available in the project repository, subject to the consent provided by participants.

6.6 Results

Table 2 summarizes the mean ratings for the six Likert-scale items. Overall, participants evaluated Annotaterm positively. The highest-rated aspect was the usefulness of the annotation tables and highlighted preview ($M = 4.57$), suggesting that the visual feedback and monitoring mechanisms were effective. The overall suitability of the tool also received a high score ($M = 4.36$). The specific feature most closely related to the contribution of this paper, support for overlapping and nested annotations, was positively evaluated ($M = 4.00$), indicating that participants found the fragment-based interaction model useful for handling complex annotation cases. The lowest-rated item concerned ease of understanding at first use ($M = 3.57$), suggesting that onboarding and initial guidance could be improved.

Open-ended responses revealed several recurring positive themes. Participants frequently highlighted the usefulness of Annotaterm for representing discontinuous and overlapping terms, which many considered an advantage over previously used annotation tools. The highlighted preview and annotation tables were repeatedly mentioned as particularly useful for monitoring annotation progress and keeping track of stored terms. Several participants also appreciated the overall interface layout, especially the visibility of controls and the side-by-side organization of text and tools. Deletion and reset functions were explicitly mentioned as useful during annotation.

Recurring suggestions for improvement included:

Table 2 Mean ratings on a 5-point Likert scale

Item	Statement	Mean (M)
Q3	The interface was easy to understand at first use	3.57
Q4	Selecting fragments and creating annotations was straightforward	3.86
Q5	The distinction between simple and composite terms was clear	3.79
Q6	The tool made it easy to annotate overlapping/nested terms	4.00
Q7	The annotation tables and highlighted preview helped monitor work	4.57
Q8	Overall, the tool was suitable for the annotation task	4.36

- Simplifying the workflow for creating composite annotations;
- Providing clearer initial instructions or onboarding;
- Improving support for tablet/mobile devices;
- Introducing more consistent color schemes for annotation types or labels; and
- Improving label consistency through predefined labels or controlled vocabularies.

Some participants also reported initial difficulty in selecting precise text fragments, particularly when creating overlapping or nested annotations.

6.7 Discussion of the evaluation

The evaluation does not aim to provide a large-scale benchmark of annotation performance. Instead, it offers an initial assessment of whether the proposed interaction model is understandable and usable in realistic annotation scenarios.

The results are encouraging. Participants generally perceived Annotaterm as suitable for terminology-oriented annotation tasks, particularly appreciating its support for discontinuous and overlapping expressions and the effectiveness of its visual monitoring components.

At the same time, the study identified areas for refinement. In particular, improvements in onboarding, simplification of composite-term creation, and better support for non-desktop devices may further increase usability. These observations are valuable because they connect the conceptual advantages of the fragment-based model with the practical requirements of annotation work in digital-library and terminology-oriented settings.

7 Conclusion and future work

This paper addressed the problem of annotating discontinuous, overlapping, and partially shared textual expressions in terminology-oriented and digital-library workflows. While many existing annotation environments provide strong support for contiguous span annotation and relational modeling, discontinuous structures are often represented indirectly through workaround mechanisms that increase annotation complexity and reduce the transparency of the resulting annotation objects.

To address this limitation, we introduced a fragment-based model in which annotation objects are represented as conceptual units composed of one or more explicitly stored textual fragments linked under a shared identifier. This model supports simple, composite, and overlapping annotations in a uniform and inspectable way, preserving both conceptual coherence and exportability.

We operationalized this model through *Annotaterm*, a lightweight web-based annotation environment implemented in R Shiny. Annotaterm supports direct fragment selection, aggregation of discontinuous expressions, overlapping and nested annotations, visual highlighting, tabular inspection, and export in interoperable lightweight formats. Compared with more general-purpose annotation platforms, the system prioritizes transparency, ease of deployment, and suitability for small- to medium-scale terminology and digital-library annotation tasks.

A preliminary user evaluation involving 14 participants provided encouraging results. Participants generally perceived the tool as suitable for annotation tasks and particularly appreciated the highlighted preview, annotation tables, and support for discontinuous and overlapping expressions. The evaluation also identified areas for improvement, including clearer onboarding, simplification of composite annotation workflows, and better support for non-desktop devices.

Future work will focus on extending the system in several directions. Planned developments include improved editing and revision workflows, stronger normalization and controlled-label support, richer export formats for interoperability with existing annotation ecosystems, and collaborative or multi-user annotation features. Further large-scale evaluations will also be needed to assess annotation efficiency, agreement, and integration in real digital-library and terminology-management scenarios.

Overall, the proposed fragment-based approach contributes both a conceptual model and a practical annotation environment for the transparent representation of complex textual structures in annotation workflows.

Acknowledgements This work has been partially supported by the HEREDITARY Project, as part of the European Union's Horizon Europe research and innovation programme under grant agreement No GA 101137074, and it is part of the initiatives of the Center for Studies in Computational Terminology (CENTRICO) of the University of Padua and in the research directions of the Italian Common Language Resources and Technology Infrastructure CLARIN-IT.

Funding Open access funding provided by Università degli Studi di Padova within the CRUI-CARE Agreement.

References

1. Agosti, M., Ferro, N., Frommholz, I., Thiel, U.: Annotations in digital libraries and collaboratories-facets, models and usage. In: Heery, R., Lyon, L. (eds.) Research and Advanced Technology for Digital Libraries, pp. 244–255. Springer, Berlin, Heidelberg (2004). https://doi.org/10.1007/978-3-540-30230-8_23
2. Giovannetti, E., Albanesi, D., Bellandi, A., Marchi, S., Papini, M., Sciolette, F.: Maia: an open collaborative platform for text annotation, e-lexicography, and lexical linking. Um. Digit. **18**, 27–52 (2024). https://doi.org/10.6092/issn.2532-8816/19705
3. Alhassan, A., Schlegel, V., Aloud, M., Batista-Navarro, R., Nenadic, G.: Discontinuous named entities in clinical text: a systematic literature review. J. Biomed. Inform. **162**, 104783 (2025). https://doi.org/10.1016/j.jbi.2025.104783
4. Colucci Cante, L., D'Angelo, S., Di Martino, B., Graziano, M.: Text annotation tools: a comprehensive review and comparative analysis. In: Barolli, L. (ed.) Complex, Intelligent and Software Intensive Systems, pp. 353–362. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-70011-8_33
5. Jehangir, B., Radhakrishnan, S., Agarwal, R.: A survey on named entity recognition—datasets, tools, and methodologies. Nat. Lang. Process. J. **3**, 100017 (2023). https://doi.org/10.1016/j.nlp.2023.100017
6. Lever, J., Altman, R., Kim, J.-D.: Extending TextAE for annotation of non-contiguous entities. Genomics Inform. **18**(2), e15 (2020). https://doi.org/10.5808/GI.2020.18.2.e15
7. Xu, K., Feng, Y., Li, Q., Dong, Z., Wei, J.: Survey on terminology extraction from texts. J. Big Data **12**(1), 29 (2025). https://doi.org/10.1186/s40537-025-01077-x
8. Di Nunzio, G.M., Vezzani, F.: Annotaterm: A fragment-based approach for annotating discontinuous named entities and terms. In: Baraldi, L., Cornia, M., Carrara, F., Cuculo, V., Daquino, M., Marchesin, S., Paolanti, M., Sarto, S. Proceedings of the 22nd Conference on Information and Research Science Connecting to Digital and Library Science (IRCDL 2026). CEUR Workshop Proceedings. CEUR-WS.org, Modena, Italy (2026). To appear. https://github.com/gmdn/IRCDL2026
9. Corro, C., Al-Onaizan, Y., Bansal, M., Chen, Y.-N.: A fast and sound tagging method for discontinuous named-entity recognition. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pp. 19506–19518. Association for Computational Linguistics, Miami, Florida, USA (2024). https://doi.org/10.18653/v1/2024.emnlp-main.1087
10. Stenetorp, P., Pyysalo, S., Topić, G., Ohta, T., Ananiadou, S., Tsujii, J.: Brat: A web-based tool for NLP-assisted text annotation. In: Segond, F. (ed.) Proceedings of the Demonstrations at the 13th Conference of the European Chapter of the Association for Computational Linguistics, pp. 102–107. Association for Computational Linguistics, Avignon, France (2012)
11. Klie, J.-C., Bugert, M., Boullosa, B., Eckart de Castilho, R., Gurevych, I.: The INCEpTION platform: machine-assisted and knowledge-oriented interactive annotation. In: Zhao, D. (ed.) Proceedings of the 27th International Conference on Computational Linguistics: System Demonstrations, pp. 5–9. Association for Computational Linguistics, Santa Fe, New Mexico (2018)
12. He, H., Fu, S., Wang, L., Liu, S., Wen, A., Liu, H.: MedTator: A serverless annotation tool for corpus development. Bioinformatics **38**(6), 1776–1778 (2022). https://doi.org/10.1093/bioinformatics/btab880
13. Comeau, D.C., Batista-Navarro, R.T., Dai, H.-J., Islamaj Doğan, R., Jimeno Yepes, A., Khare, R., Lu, Z., Marques, H., Mattingly, C.J., Neves, M., Peng, Y., Rak, R., Rinaldi, F., Tsai, R.T.-H., Verspoor, K., Wiegers, T.C., Wu, C.H., Wilbur, W.J.: BioC interoperability track overview. Database **2014**, 053 (2014). https://doi.org/10.1093/database/bau053
14. Irrera, O., Marchesin, S., Silvello, G.: MetaTron: Advancing biomedical annotation empowering relation annotation and collaboration. BMC Bioinform. **25**(1), 112 (2024). https://doi.org/10.1186/s12859-024-05730-9
15. Tjong Kim Sang, E.F., Veenstra, J.: Representing Text Chunks. In: Thompson, H.S., Lascarides, A. Ninth Conference of the European Chapter of the Association for Computational Linguistics, pp. 173–179. Association for Computational Linguistics, Bergen, Norway (1999)
16. Shin, S.-Y., Kim, S., Wilbur, W.J., Kwon, D.: BioC viewer: a web-based tool for displaying and merging annotations in BioC. Database **2006**, 106 (2016). https://doi.org/10.1093/database/baw106